

2400031810

S. LALITH ADITYA

GITHUB: <https://github.com/lalithdev/2400031810-FSAD20252026EVENSEM-SKILL-EXPERIMENTS.git>

#SKILL - 10 📄 React State Management using useState Object

Prerequisites:

- Basic Idea of JavaScript
- Knowledge of React fundamentals

An online academic portal wants to maintain a list of students where the user can add new students, display the list, and delete a student instantly without refreshing the page. The application should use React Hooks to manage data at the component level so that the UI updates automatically whenever the state changes.

Your task is to implement a React component using useState to store an object array, add new items, display them, and delete them dynamically.

1. Create a new React component named StudentManager.

```

1  import React, { useState } from "react";
2  import "./StudentManager.css";
3
4  function StudentManager() {
5
6      const [students, setStudents] = useState([
7          { id: 1, name: "Rahul", course: "CSE" },
8          { id: 2, name: "Anita", course: "ECE" },
9          { id: 3, name: "Kiran", course: "IT" },
10         { id: 4, name: "Meena", course: "AI" },
11         { id: 5, name: "Arjun", course: "DS" }
12     ]);
13
14     const [newStudent, setNewStudent] = useState({
15         id: "",
16         name: "",
17         course: ""
18     });
19
20     const handleChange = (e) => {
21         setNewStudent({
22             ...newStudent,
23             [e.target.name]: e.target.value
24         });
25     };
26
27     const addStudent = () => {
28         if (!newStudent.id || !newStudent.name || !newStudent.course) return;
29
30         setStudents([...students, newStudent]);
31
32         setNewStudent({
33             id: "",
34             name: "",
35             course: ""
36         });
37     };
38
39     const deleteStudent = (id) => {
40         const updatedStudents = students.filter(student => student.id !== id);

```

2. Inside the component, create an initial students array with at least 5 objects containing:
 - a. id
 - b. name
 - c. course

```

<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Course</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody>
    {students.map((student) => (
      <tr key={student.id}>
        <td>{student.id}</td>
        <td>{student.name}</td>
        <td>{student.course}</td>
        <td>
          <button
            className="delete"
            onClick={() => deleteStudent(student.id)}
          >
            Delete
          </button>
        </td>
      </tr>
    ))}
  </tbody>
</table>

```

3. Use useState to store:
 - a. students (array of student objects)

b. newStudent (object with fields like id, name, course)

```
const [students, setStudents] = useState([
  { id: 1, name: "Rahul", course: "CSE" },
  { id: 2, name: "Anita", course: "ECE" },
  { id: 3, name: "Kiran", course: "IT" },
  { id: 4, name: "Meena", course: "AI" },
  { id: 5, name: "Arjun", course: "DS" }
]);

const [newStudent, setNewStudent] = useState({
  id: "",
  name: "",
  course: ""
});
```

Create input boxes to accept id, name, and course.

i. Update the newStudent object using onChange events.

```

const handleChange = (e) => {
  setNewStudent({
    ...newStudent,
    [e.target.name]: e.target.value
  });
};

const addStudent = () => {
  if (!newStudent.id || !newStudent.name || !newStudent.course) return;

  setStudents([...students, newStudent]);

  setNewStudent({
    id: "",
    name: "",
    course: ""
  });
};

const deleteStudent = (id) => {
  const updatedStudents = students.filter(student => student.id !== id);
  setStudents(updatedStudents);
};

```

Create an Add Student button.

- i. On click, add the newStudent object to the students array.
- ii. After adding, clear the input fields.

```

setNewStudent({
  id: "",
  name: "",
  course: ""
});

```

Display the list of students below using the students state.

Each student row should have a Delete button.

- i. On clicking delete, remove the record from the students array.
- ii. The UI should update immediately using setStudents.

Student Manager

Student ID	Student Name	Course
Add Student		

ID	Name	Course	Action
3	Kiran	IT	Delete
4	Meena	AI	Delete
5	Arjun	DS	Delete
249043	bnje	bfjr	Delete

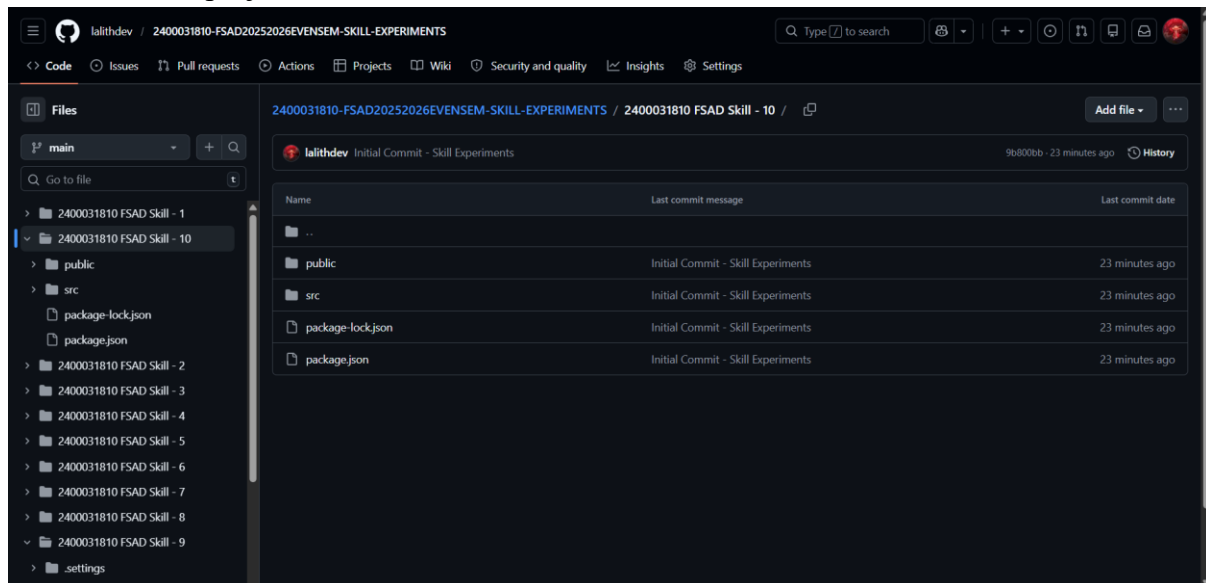
If the list becomes empty, display the message:

"No students available"

Add basic CSS styling for a clean UI (margin, padding, table style, buttons).

```
5      font-family:Arial;
6    }
7
8    .form{
9      margin-bottom:20px;
10   }
11
12   input{
13     margin:5px;
14     padding:8px;
15   }
16
17   button{
18     padding:8px 12px;
19     margin:5px;
20     cursor:pointer;
21   }
22
23   table{
24     width:100%;
25     border-collapse:collapse;
26   }
27
28   th,td{
29     border:1px solid #ccc;
30     padding:8px;
31     text-align:center;
32   }
33
34   th{
35     background: #f2f2f2;
36   }
37
38   .delete{
39     background: red;
40     color: white;
41     border:none;
42   }
```

Push the React project to GitHub.



VIVA QUESTIONS:

1. What is the purpose of the useState hook in React?

useState is used to create and manage state variables in functional components. It allows React components to store and update data dynamically.

2. How does React update the UI when state changes?

When setState (or setStudents) is called, React re-renders the component and updates the UI automatically based on the new state.

3. Why do we store form inputs in state for controlled components?

Storing inputs in state allows React to control the form elements. This makes validation, dynamic updates, and form handling easier.

4. What is the importance of using keys when rendering lists?

Keys help React identify which elements changed, were added, or removed. This improves performance and prevents UI rendering errors.

5. Can one component use multiple useState variables? Why?

Yes. A component can use multiple useState variables to manage different pieces of state independently (e.g., students, newStudent, search, etc.).